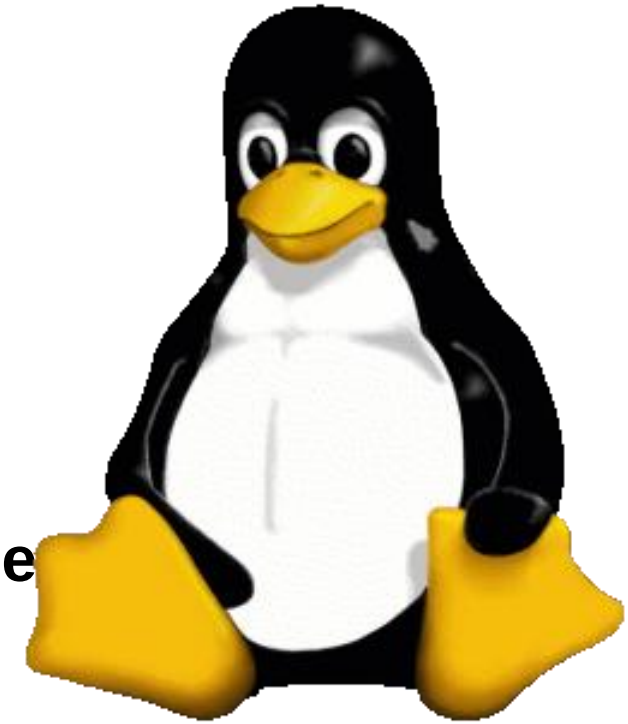
Linux Kernel

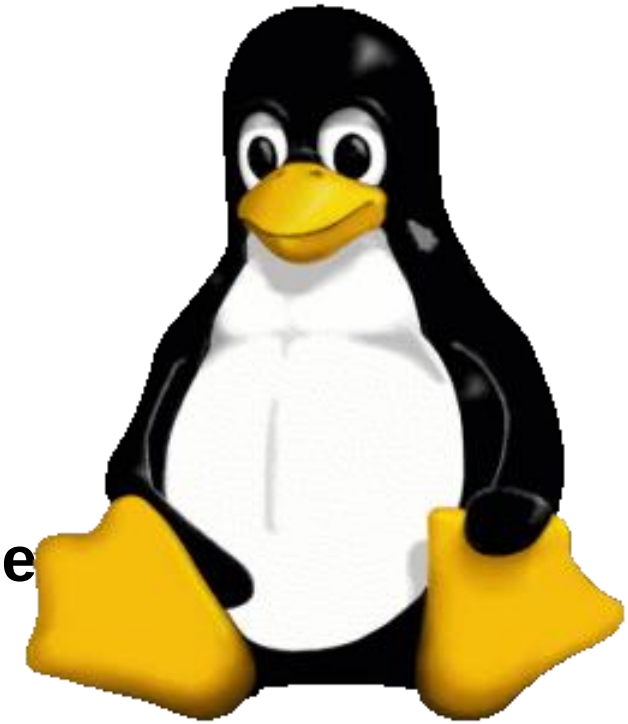


Chentao Wu吴晨涛
Professor
School of Computer Science
wuct@cs.sjtu.edu.cn

上海交通大學

2. Linux Kernel Programming Basic

Chentao Wu
Professor
School of Computer Science
wuct@cs.sjtu.edu.cn



上海交通大學

Outline

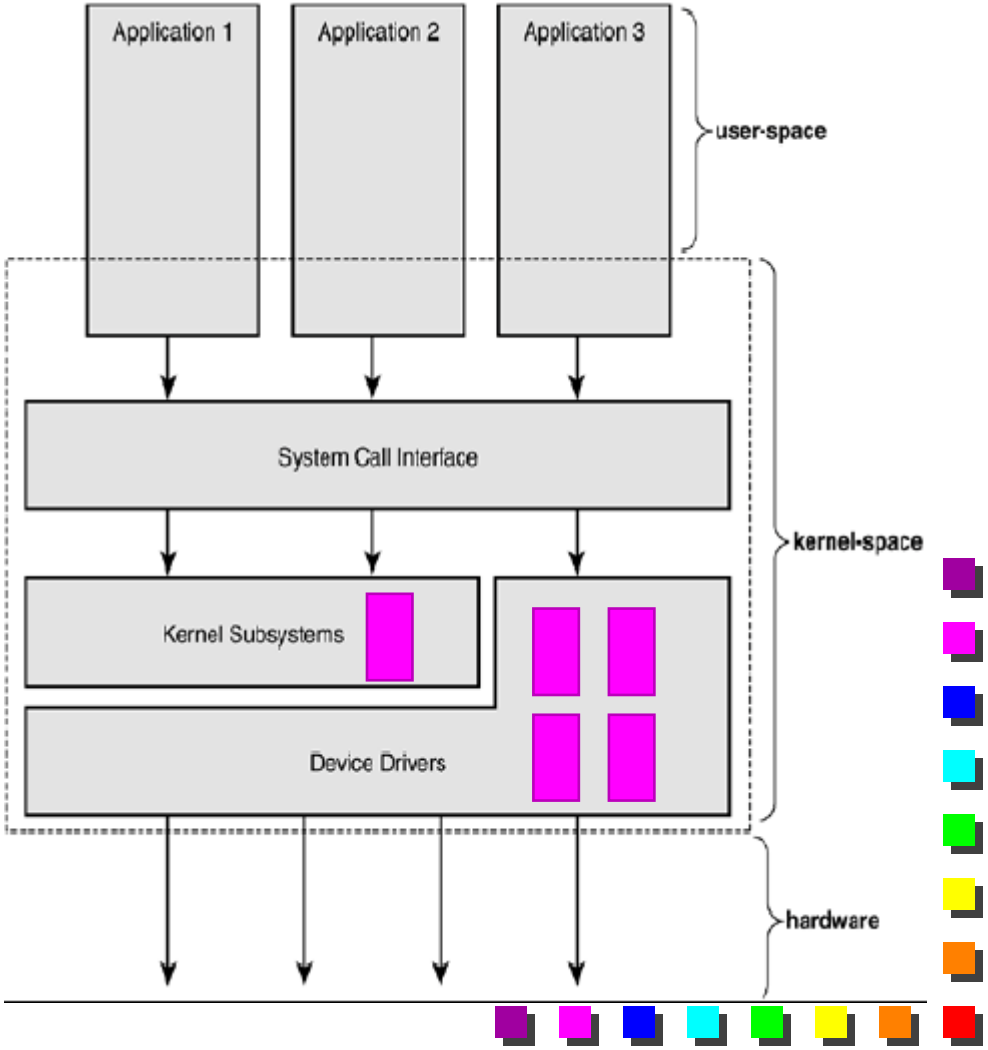
- **Linux Kernel Programming Basic**
- Introduction of the /proc File System
- Module Program in /proc File System

Linux Kernel Programming

- What makes kernel programming different?
  - OS management
    - Process management, memory management
    - File systems
  - Types of devices
    - (Char, Block, SCSI, Net)-based devices → device drivers
  - Loaded as modules or static in the kernel
  - Challenges
    - Portability
    - IPC (Inter-Process Communication)
    - Hardware Management
    - Interface Stability

Module

- What's a kernel module?
  - (wiki) An object file that contains code to extend the running kernel;
  - (RedHat) Modules are pieces of code that can be loaded and unloaded into the kernel upon demand.
  - Usage
  - Most current UNIX-like and Microsoft Windows Systems

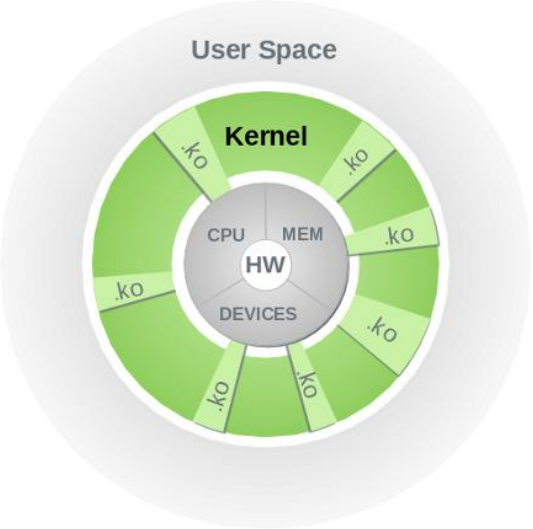


上海交通大學

Advantages and Disadvantages

- Why module? (Advantages)
  - Allowing the dynamic insertion and removal of code from the kernel at run-time.
  - Save memory cost
- Minor Disadvantage
  - Fragmentation Penalty → decrease memory performance

Current Kernel Modules



- # cd /lib/modules/2.6.32-22-generic/
- # find . -name "*.ko"

./kernel/arch/x86/kernel/microcode.ko

./kernel/arch/x86/kernel/msr.ko

./kernel/arch/x86/kernel/cpuid.ko

./kernel/arch/x86/kernel/cpu/mcheck/mce-xeon75xx.ko

./kernel/arch/x86/kernel/cpu/mcheck/mce-inject.ko

./kernel/arch/x86/kernel/cpu/cpufreq/p4-clockmod.ko

./kernel/arch/x86/kernel/cpu/cpufreq/speedstep-lib.ko

./kernel/arch/x86/kvm/kvm.ko

... and on and on and on ...

Current Loadable Modules

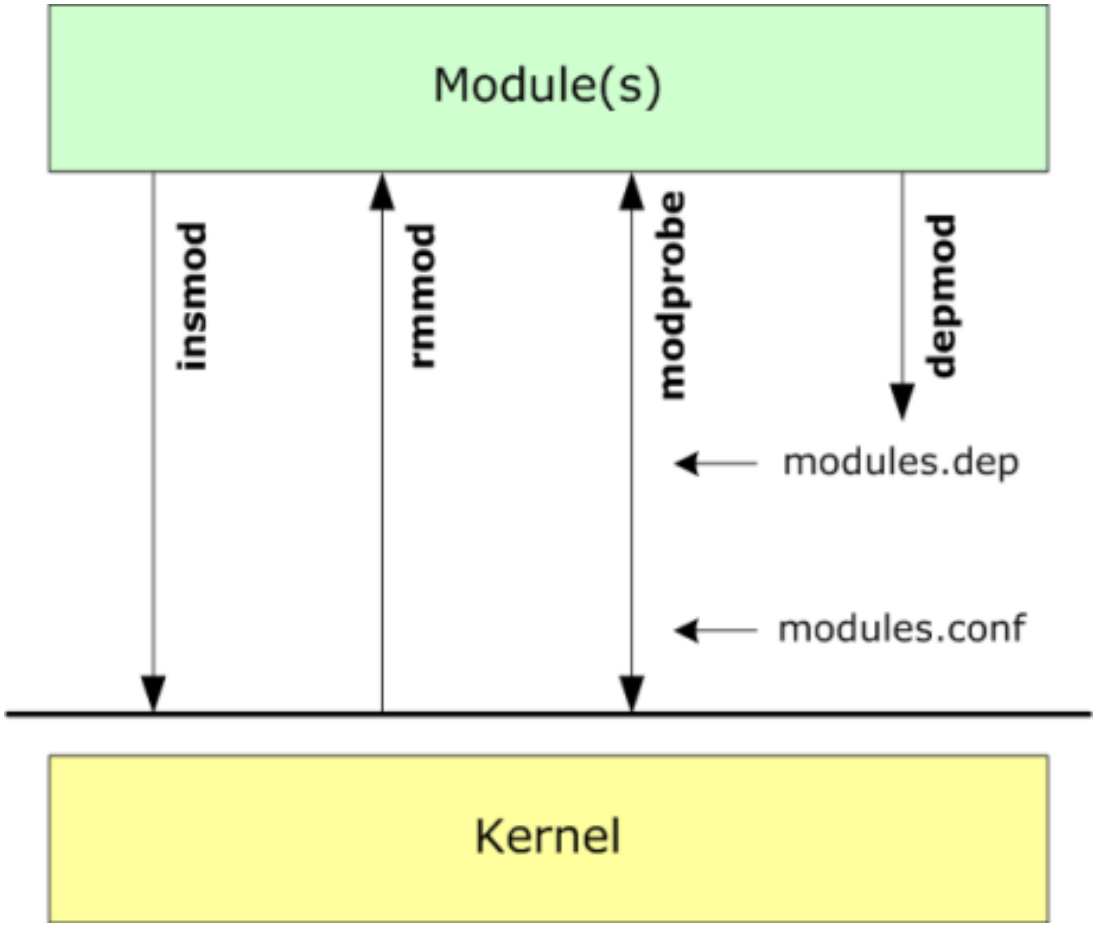
- # lsmod

```
Module                  Size    Used by
cryptd                  8116      0
aes_x86_64              7912      1
aes_generic            27607      1
aes_x86_64 nls_iso8859_1  4633    1
nls_cp437               6351      1
vfat                   10802      1
fat                    55350      1vfat
binfmt_misc             7960      1
ppdev                   6375      0
... etc. ...
```

- # cat /proc/modules

上海交通大學

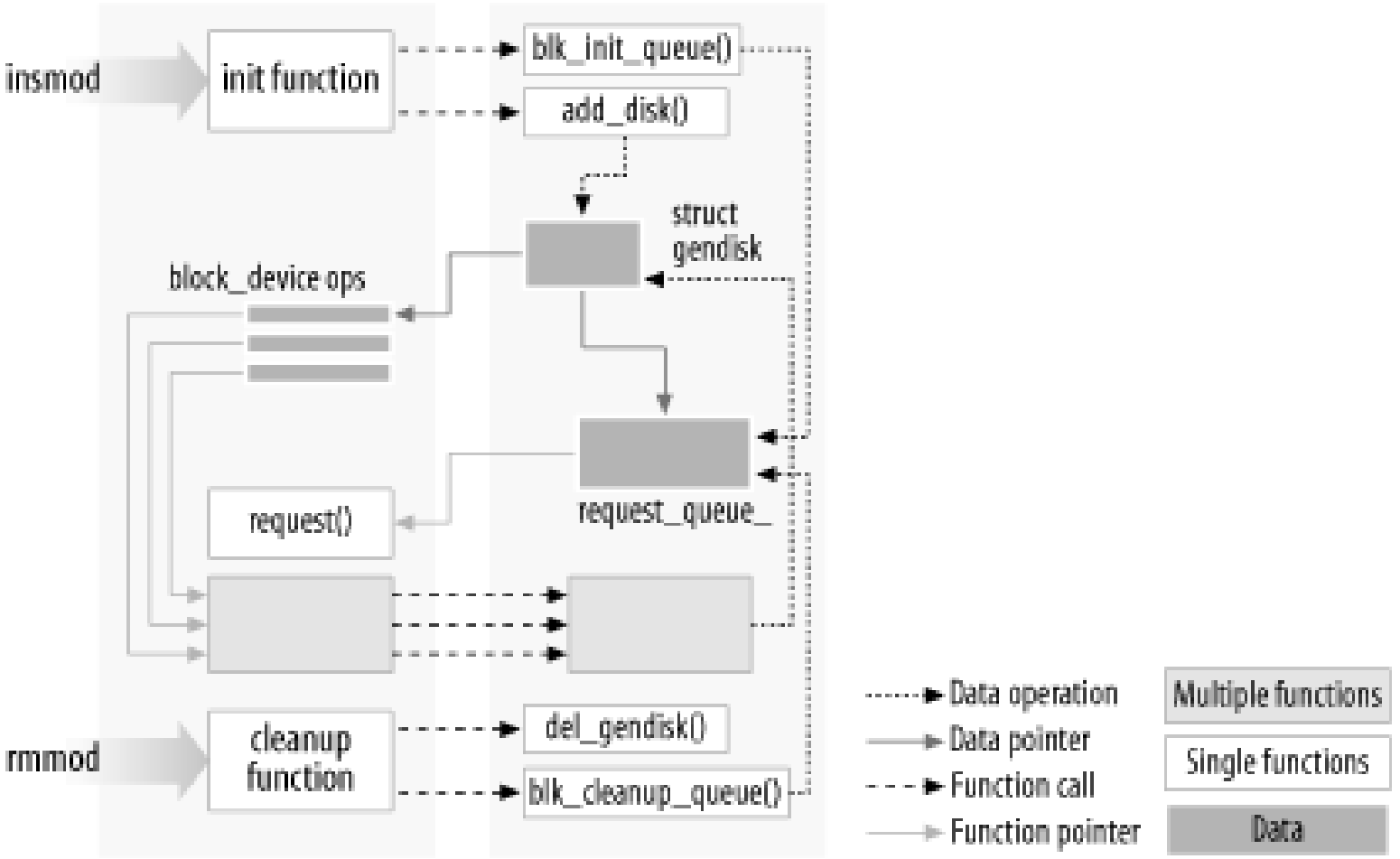
Module Commands



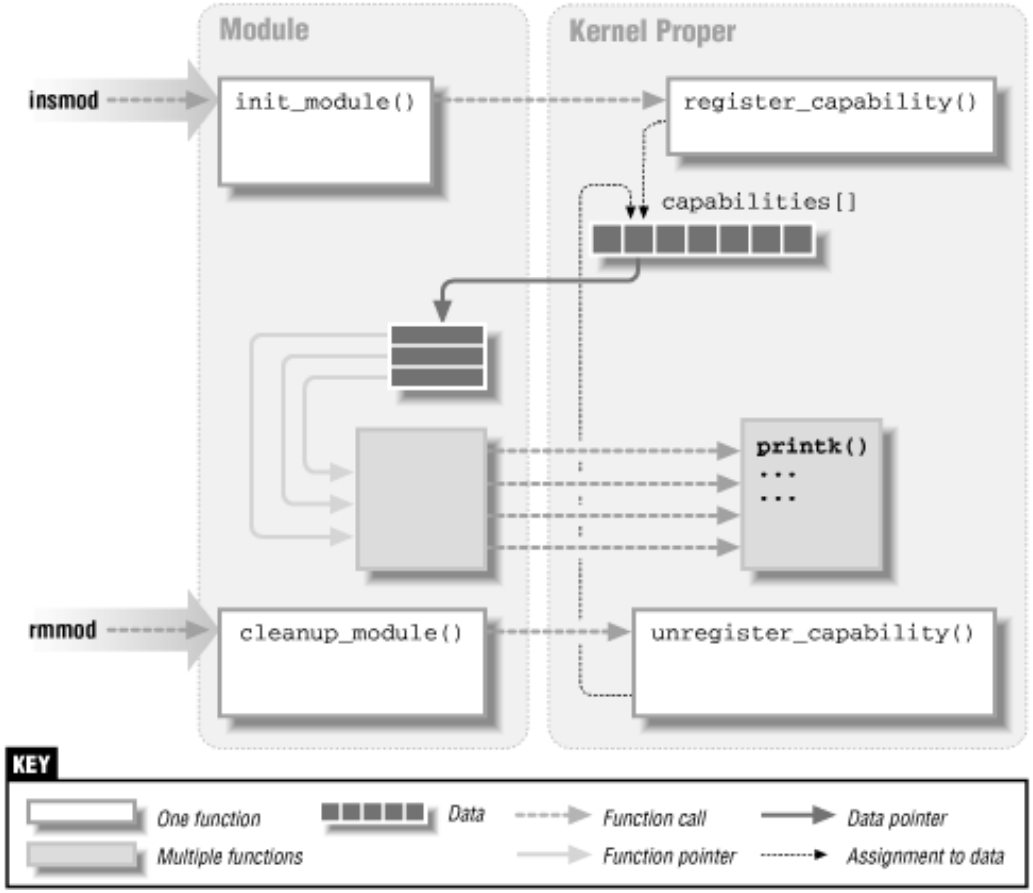
Commands For Modules

- #insmod hello.ko /*insert a module*/
- #rmmod hello.ko /*delete a module*/
- #lsmod /*list the current modules*/
- #modinfo hello.ko /*list the information of a module*/
- #modprobe /*insert or delete a module, and handle the dependencies between modules*/
- #depmod /*map interdependencies between modules*/

Example: A Module Design



Example: Linking A Module



Printk (1)

- printk(KERN_INFO "Message: %s\n", arg);

Console Log levels

Name	String	Alias function
KERN_EMERG	“0”	pr_emerg()
KERN_ALERT	“1”	pr_alert()
KERN_CRIT	“2”	pr_crit()
KERN_ERR	“3”	pr_err()
KERN_WARNING	“4”	pr_warn()
KERN_NOTICE	“5”	pr_notice()
KERN_INFO	“6”	pr_info()
KERN_DEBUG	“7”	pr_debug() and pr_devel() if DEBUG is defined
KERN_DEFAULT	“”	
KERN_CONT	“c”	pr_cont()

Printk (2)

- Get current console_level

```
$ cat /proc/sys/kernel/printk
   4       4       1       7
```

current, *default*, *minimum* and *boot-time-default* log levels

dmesg command (1)

- The dmesg command allows you to view messages from the kernel ring buffer. These messages can be useful for troubleshooting issues with the kernel, drivers, or other system components.
- To use dmesg, simply type the command followed by any options or arguments you want to use. Some common options include:

```
dmesg [options]
```

dmesg command (2)

- -C: It clears the ring buffer.
- -c: Print and clear the ring buffer.
- -H: It provides the human-readable output, making timestamps and information more understandable
- -T: Displays timestamps in a human-readable format for each message.
- -l: Filters messages by their log level, such as errors, warnings, or info.
- -k: Displays only kernel-related messages, hiding user-space messages.
- -s: Specifies how much buffer to read for messages, such as 1024 bytes.
- -f: Filters messages by their facility, like kernel or user messages.
- -r: Shows the raw output without converting priority info into readable text.
- -x: Converts the facility and log level into readable text, like “kernel.info”.

dmesg command (3)

```
anees@ultahost:~$ sudo dmesg
[sudo] password for anees:
[    0.000000] Linux version 6.8.0-47-generic (buildd@lcy02-amd64-023) (x86_64-l
inux-gnu-gcc-13 (Ubuntu 13.2.0-23ubuntu4) 13.2.0, GNU ld (GNU Binutils for Ubunt
u) 2.42) #47-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep 27 21:40:26 UTC 2024 (Ubuntu 6.8
.0-47.47-generic 6.8.12)
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-47-generic root=UUID
=15f9ffef-11c3-464f-84c7-71b5201ca3b4 ro security=selinux quiet splash
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Hygon HygonGenuine
[    0.000000]   Centaur CentaurHauls
[    0.000000]   zhaoxin   Shanghai
[    0.000000] BIOS-provided physical RAM map:
[    0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[    0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[    0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000dffeffff] usable
[    0.000000] BIOS-e820: [mem 0x00000000dfff0000-0x00000000dfffffff] ACPI data
[    0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000fee00000-0x00000000fee00fff] reserved
```

dmesg command (4)

- If you want to add colors to make the messages easier to understand, you can colorize them with the -L option:

```
sudo dmesg -L
```

```
tclass=netlink_kobject_uevent_socket permissive=1
[80989.779668] audit: type=1400 audit(1729872576.914:803): avc:  denied  { getattr } for  pid=10
768 comm="fprintd" scontext=system_u:system_r:fprintd_t:s0 tcontext=system_u:system_r:fprintd_t:
s0 tclass=netlink_kobject_uevent_socket permissive=1
[80989.785424] audit: type=1400 audit(1729872576.920:804): avc:  denied  { read } for  pid=10768
 comm="fprintd" name="c189:0" dev="tmpfs" ino=833 scontext=system_u:system_r:fprintd_t:s0 tconte
xt=system_u:object_r:udev_runtime_t:s0 tclass=file permissive=1
[80989.785430] audit: type=1400 audit(1729872576.920:805): avc:  denied  { open } for  pid=10768
 comm="fprintd" path="/run/udev/data/c189:0" dev="tmpfs" ino=833 scontext=system_u:system_r:fpri
ntd_t:s0 tcontext=system_u:object_r:udev_runtime_t:s0 tclass=file permissive=1
[80989.786130] audit: type=1400 audit(1729872576.921:806): avc:  denied  { getattr } for  pid=10
768 comm="fprintd" path="/run/udev/data/c189:0" dev="tmpfs" ino=833 scontext=system_u:system_r:f
printd_t:s0 tcontext=system_u:object_r:udev_runtime_t:s0 tclass=file permissive=1
[81022.295222] kauditd_printk_skb: 4 callbacks suppressed
[81022.295226] audit: type=1400 audit(1729872609.433:811): avc:  denied  { read write } for  pid
=10814 comm="mount" name="syscheck-squashfs-333296820" dev="sda2" ino=1179992 scontext=system_u:
system_r:mount_t:s0 tcontext=system_u:object_r:tmp_t:s0 tclass=file permissive=1
[81022.295231] audit: type=1400 audit(1729872609.433:812): avc:  denied  { open } for  pid=10814
 comm="mount" path="/tmp/syscheck-squashfs-333296820" dev="sda2" ino=1179992 scontext=system_u:s
ystem_r:mount_t:s0 tcontext=system_u:object_r:tmp_t:s0 tclass=file permissive=1
[81022.294951] loop4: detected capacity change from 0 to 8
[81022.295652] SELinux: security_context_str_to_sid (system_u:object_r:snappy_snap_t:s0) failed
with errno=-22
anees@ultahost:~$
```

dmesg command (5)

- We can use the dmesg command with the head command to show a limited/specific number of kernel messages from the top:

```
sudo dmesg | head -15
```

```
anees@ultahost:~$ sudo dmesg | head -15
[    0.000000] Linux version 6.8.0-47-generic (buildd@lcy02-amd64-023) (x86_64-l
inux-gnu-gcc-13 (Ubuntu 13.2.0-23ubuntu4) 13.2.0, GNU ld (GNU Binutils for Ubunt
u) 2.42) #47-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep 27 21:40:26 UTC 2024 (Ubuntu 6.8
.0-47.47-generic 6.8.12)
[    0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.8.0-47-generic root=UUID
=15f9ffef-11c3-464f-84c7-71b5201ca3b4 ro security=selinux quiet splash
[    0.000000] KERNEL supported cpus:
[    0.000000]   Intel GenuineIntel
[    0.000000]   AMD AuthenticAMD
[    0.000000]   Hygon HygonGenuine
[    0.000000]   Centaur CentaurHauls
[    0.000000]   zhaoxin   Shanghai
[    0.000000] BIOS-provided physical RAM map:
[    0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[    0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[    0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000dffeffff] usable
[    0.000000] BIOS-e820: [mem 0x00000000dfff0000-0x00000000dfffffff] ACPI data
[    0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec00fff] reserved
anees@ultahost:~$
```

dmesg command (6)

- Similarly, we can use the tail command with dmesg to show the last N kernel messages:

```
sudo dmesg | tail -15
```

```
etworkManager_t:s0 tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=dir permissive=1
[  612.792062] audit: type=1400 audit(1729988270.640:229): avc:  denied  { add_name } for
pid=658 comm="NetworkManager" name="3.C2WWV2" scontext=system_u:system_r:NetworkManager_t:s
0 tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=dir permissive=1
[  612.792064] audit: type=1400 audit(1729988270.640:230): avc:  denied  { create } for  pi
d=658 comm="NetworkManager" name="3.C2WWV2" scontext=system_u:system_r:NetworkManager_t:s0
tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=file permissive=1
[  612.792065] audit: type=1400 audit(1729988270.640:231): avc:  denied  { remove_name } fo
r  pid=658 comm="NetworkManager" name="3.C2WWV2" dev="tmpfs" ino=1867 scontext=system_u:sys
tem_r:NetworkManager_t:s0 tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=dir permiss
ive=1
[  612.792079] audit: type=1400 audit(1729988270.640:232): avc:  denied  { rename } for  pi
d=658 comm="NetworkManager" name="3.C2WWV2" dev="tmpfs" ino=1867 scontext=system_u:system_r
:NetworkManager_t:s0 tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=file permissive=
1
[  612.792081] audit: type=1400 audit(1729988270.640:233): avc:  denied  { unlink } for  pi
d=658 comm="NetworkManager" name="3" dev="tmpfs" ino=1827 scontext=system_u:system_r:Networ
kManager_t:s0 tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=file permissive=1
[  612.792082] audit: type=1400 audit(1729988270.640:234): avc:  denied  { read } for  pid=
658 comm="NetworkManager" name="devices" dev="tmpfs" ino=1230 scontext=system_u:system_r:Ne
tworkManager_t:s0 tcontext=system_u:object_r:initrc_runtime_t:s0 tclass=dir permissive=1
[  633.200497] kauditd_printk_skb: 1 callbacks suppressed
[  633.200501] audit: type=1400 audit(1729988291.049:236): avc:  denied  { execmem } for  p
id=2937 comm="gnome-shell" scontext=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 t
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 tclass=process permissive=1
anees@ultahost:~$
```

A Simple Module

```
#include <linux/kernel.h>   /* Needed for KERN_INFO */
#include <linux/module.h>  /* Needed by all modules */
#include <linux/init.h>  /*Needed for init and cleanup functions*/
```

Entrance→

```
static int __init hello_init(void)
{
printk(KERN_INFO “Hello, world\n");
return 0;
}
```

Exit→

```
static void __exit hello_exit(void)
{
printk(KERN_INFO "Goodbye, world\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

loglevels:

KERN_EMERG = “<0>”
KERN_ALERT
KERN_CRIT
KERN_ERR
KERN_WARNING
KERN_NOTICE
KERN_INFO
KERN_DEBUG

A Real Module in VFS (Minix FS)

```
/* linux/fs/minix/inode.c
 ......*/
#include <linux/module.h>
#include "minix.h"
#include <linux/buffer_head.h>
#include <linux/slab.h>
#include <linux/init.h>
#include <linux/vfs.h>
......
static struct file_system_type minix_fs_type
= {
.owner = THIS_MODULE,
.name = "minix",
.get_sb = minix_get_sb,
.kill_sb = kill_block_super,
.fs_flags = FS_REQUIRES_DEV,
};
```

```
static int __init init_minix_fs(void)
{
int err = init_inodecache();
if (err)
goto out1;
err = register_filesystem(&minix_fs_type);
if (err)
goto out;
return 0;
out:
destroy_inodecache();
out1:
return err;
}
static void __exit exit_minix_fs(void)
{
unregister_filesystem(&minix_fs_type);
destroy_inodecache();
}
```

上海交通大學

Compiling Modules (1)

- Makefile
  - **obj-m:=hello.o**
- Multiple source files
  - **obj-m:=hello.o**
  - **hello-objs := a.o b.o**

hello.c→hello.o

```
obj-m:=hello.o
all:
	make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules
clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean
```

This command starts by changing its directory to the one provided with the –C option (that is, your kernel source directory). There it finds the kernel's top-level makefile. The M= option causes that makefile to move back into your module source directory before trying to build the modules target. This target, in turn, refers to the list of modules found in the obj-m variable.

-- Linux Device Drivers (3rd Edition)

Compiling Modules (2)

- Run the Module

Command:

```
# make
# make clean
```

Another Type of Makefile

homework1.c→homework

```
obj-m := homework1.o
KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)
all:
	make -C $(KDIR) M=$(PWD) modules
clean:
	rm *.o *.ko *.mod.c Module.symvers
modules.order -f
```

Results

carlos@carlos-VirtualBox: ~/hello-world-linux-module

```
carlos@carlos-VirtualBox:~/hello-world-linux-module$ ls
lkm_hello_world.c  Makefile
carlos@carlos-VirtualBox:~/hello-world-linux-module$ make
make -C /lib/modules/4.4.0-31-generic/build M=/home/carlos/hello-world-linux-module modules
make[1]: Entering directory '/usr/src/linux-headers-4.4.0-31-generic'
  CC [M]  /home/carlos/hello-world-linux-module/lkm_hello_world.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/carlos/hello-world-linux-module/lkm_hello_world.mod.o
  LD [M]  /home/carlos/hello-world-linux-module/lkm_hello_world.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.4.0-31-generic'
carlos@carlos-VirtualBox:~/hello-world-linux-module$ make test
sudo dmesg -C
sudo insmod lkm_hello_world.ko
sudo rmmod lkm_hello_world.ko
dmesg
[ 1837.445361] Hello, World!
[ 1837.461268] Goodbye, World!
carlos@carlos-VirtualBox:~/hello-world-linux-module$
```

Module Parameters

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
#include <linux/moduleparam.h>
static int test;
module_param(test, int, 0644);
void hello_foo(void)
{
printk("Hello\n");
}
EXPORT_SYMBOL(hello_foo);

static int __init hello_init(void)
{
printk(KERN_INFO "Hello world\n");
printk(KERN_INFO "Params:
    test:%d;\n",test);
return 0;
}
static void __exit hello_exit(void)
{
printk(KERN_INFO "Goodbye
    world\n");
}
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Test");
MODULE_AUTHOR("xxx");
module_init(hello_init);
module_exit(hello_exit);
```

```
#insmod hello.ko test=10
```

上海交通大學

Outline

- Linux Kernel Programming Basic
- **Introduction of the /proc File System**
- Module Program in /proc File System

The /proc

- Blue represents subdirectories. Some are numbered which is a representation of a specific process.
- White represents normal files containing data e.g, memory statistics, cpu information etc.

```
lumunge@opengenus:~$ ls /proc/
1     126   167   20    230   265   301   425   800   irq
10    127   168   2010  231   266   302   426   801   kallsyms
100   128   1682  2019  232   267   303   427   802   kcore
101   129   1697  2030  233   268   304   428   803   keys
102   13    1698  2033  234   269   305   429   804   key-users
1022  130   17    2035  235   27    306   43    808   kmsg
103   131   1703  2039  236   270   307   430   833   kpagecgroup
1038  1317  1705  2040  237   271   308   431   9     kpagecount
104   132   1709  2041  238   272   309   432   93    kpageflags
1050  133   1712  2042  239   273   31    433   939   loadavg
1052  1332  1717  2043  24    274   310   434   94    locks
106   134   1726  2047  240   275   311   436   95    mdstat
107   135   173   2053  241   276   312   437   96    meminfo
109   136   1743  2058  242   277   313   438   965   misc
11    137   1750  2059  243   278   317   439   97    modules
110   138   1752  2063  244   279   32    44    974   mounts
111   139   1757  2064  245   28    33    443   976   mpt
112   14    1763  2068  246   280   336   45    98    mtrr
113   140   1765  2081  247   281   337   46    99    net
114   141   1769  2085  248   282   34    5     acpi  pagetypeinfo
```

cat /proc/version

```
$ cat /proc/version
```

```
$ uname -a
```

```
lumunge@opengenus:~$ cat /proc/version
Linux version 5.13.0-35-generic (buildd@ubuntu) (gcc (Ubuntu 9.3.0-17ubuntu1~20.
04) 9.3.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #40~20.04.1-Ubuntu SMP Mon Mar
 7 09:18:32 UTC 2022
lumunge@opengenus:~$ uname -a
Linux opengenus 5.13.0-35-generic #40~20.04.1-Ubuntu SMP Mon Mar 7 09:18:32 UTC
2022 x86_64 x86_64 x86_64 GNU/Linux
```

cat /proc/[process ID]

```
$ cat /proc/1800/status
```

```
lumunge@opengenus:~$ ps -a
    PID TTY          TIME CMD
   1763 tty2     00:00:03 Xorg
   1800 tty2     00:00:00 gnome-session-b
   2766 pts/0    00:00:00 ps
lumunge@opengenus:~$ cat /proc/1800/status
Name:   gnome-session-b
Umask:  0002
State:  S (sleeping)
Tgid:   1800
Ngid:   0
Pid:    1800
PPid:   1752
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 120 132 133 1000
NStgid: 1800
```

cat /proc/cpuinfo

```
$ cat /proc/cpuinfo
```

```
lumunge@opengenus:~$ cat /proc/cpuinfo
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 69
model name      : Intel(R) Core(TM) i5-4300U CPU @ 1.90GHz
stepping        : 1
microcode       : 0x26
cpu MHz         : 2494.226
cache size      : 3072 KB
physical id     : 0
siblings        : 1
core id         : 0
cpu cores       : 1
apicid          : 0
initial apicid  : 0
fpu             : yes
fpu_exception   : yes
cpuid level     : 13
```

cat /proc/meminfo

```
$ cat /proc/meminfo
```

```
lumunge@opengenus:~$ cat /proc/meminfo | head -10
MemTotal:        3982664 kB
MemFree:         1710724 kB
MemAvailable:    2506028 kB
Buffers:          108068 kB
Cached:           834280 kB
SwapCached:            0 kB
Active:           387728 kB
Inactive:        1489124 kB
Active(anon):       1764 kB
Inactive(anon):   941668 kB
lumunge@opengenus:~$ free
              total        used        free      shared  buff/cache   available
Mem:        3982664     1197068     1710888       16128     1074708     2506200
Swap:        945416           0      945416
```

上海交通大學

The /proc

- A pseudo file system
- Real time, resides in the virtual memory
- Tracks the processes running on the machine and the state of the system
- A new /proc file system is created every time your Linux machine reboots
- Highly dynamic. The size of the proc directory is 0 and the last time of modification is the last bootup time.

Other features

- /proc file system doesn't exist on any particular media.
- The contents of the /proc file system can be read by anyone who has the requisite permissions.
- Certain parts of the /proc file system can be read only by the owner of the process and of course root. (and some not even by root!!)
- The contents of the /proc are used by many utilities which grab the data from the particular /proc directory and display it.
- eg : top, ps, lspci ,dmesg etc

Tweak kernel parameters

- /proc/sys : Making changes in this directory enables you to make real time changes to certain kernel parameters.
- eg : /proc/sys/net/ipv4/ip_forward
- It has default value of "0" which can be seen using 'cat'.
- This can be changed in real time by just changing the value stored in this file from "0" to "1", thus allowing IP forwarding

Files in /proc

- buddyinfo
- cmdline
- cpuinfo
- crypto
- devices
- diskstats
- dma
- execdomains
- fb
- filesystems
- interrupts
- iomem
- ioports
- kcore
- kmsg
- loadavg
- locks
- mdstat
- meminfo
- misc
- modules
- mounts
- mtrr
- partitions
- pci
- self
- slabinfo
- stat
- swaps
- sysrq-trigger
- uptime
- version
- vmstat

Details of some files in /proc

- buddyinfo
  Contains the number of free areas of each order for the kernel buddy system
- cmdline
  Kernel command line
- cpuinfo
  Information about the processor(s).(Human readable)
- devices
  List of device drivers configured into the currently running kernel (block and character).
- dma
  Shows which DMA channels are being used at the moment.
- execdomains
  Execdomains, related to security

Details of some files in /proc

- fb

  Frame Buffer devices.

- filesystems

  Filesystems configured/supported into/by the kernel.

- interrupts

  Number of interrupts per IRQ on the x86 architecture.

- iomem

  This file shows the current map of the system's memory for its various devices

- ioports

  provides a list of currently registered port regions used for input or output communication with a device

/proc/kcore

- This file represents the physical memory of the system and is stored in the core file format.
- Unlike most /proc files, kcore does display a size. This value is given in bytes and is equal to the size of physical memory (RAM) used plus 4KB.
- Its contents are designed to be examined by a debugger, such as gdb, the GNU Debugger.
- Only the root user has the rights to view this file.

Details of some files in /proc (1)

- kmsg
  - Used to hold messages generated by the kernel. These messages are then picked up by other programs, such as `klogd`
- loadavg
  - Provides a look at load average
  - The first three columns measure CPU utilization of the last 1, 5, and 10 minute periods.
  - The fourth column shows the number of currently running processes and the total number of processes.
  - The last column displays the last process ID used.
- locks
  - Displays the files currently locked by the kernel

Details of some files in /proc (2)

- mdstat
  - contains the current information for multiple-disk, RAID configurations
- meminfo
  - One of the more commonly used /proc files
  - It reports back plenty of valuable information about the current utilization of RAM on the system
- misc
  - This file lists miscellaneous drivers registered on the miscellaneous major device, which is number 10
- modules
  - Displays a list of all modules that have been loaded by the system
- mounts
  - This file provides a quick list of all mounts in use by the system

Details of some files in /proc (3)

- mtrr
  - This file refers to the current Memory Type Range Registers (MTRRs) in use with the system
- partitions
  - Very detailed information on the various partitions currently available to the system
- pci
  - Full listing of every PCI device on your system
- slabinfo
  - Information about memory usage on the slab level
- stat
  - Keeps track of a variety of different statistics about the system since it was last restarted

Details of some files in /proc (4)

- swap
  - Measures swap space and its utilization
- uptime
  - Contains information about how long the system has on since its last
  - restart
- version
  - Tells the versions of the Linux kernel and gcc, as well as the version of
  - Red Hat Linux installed on the system.

The numerical named directories

- The various directories in /proc are the processes that were running at the instant a snapshot of the /proc file system was taken.
- The contents of all the directories are the same as these directories contain the various parameters and the status of the corresponding process.
- You have full access only to the processes that you have started.

A typical process directory

- cmdline : it contains the whole command line used to invoke the process. The contents of this file are the command line arguments with all the parameters (without formatting/spaces).
- cwd : symbolic link to the current working directory
- environ : contains all the process-specific environment variables
- exe : symbolic link of the executable
- maps : parts of the process' address space mapped to a file.

A typical process directory(contd.)

- fd : this directory contains the list file descriptors as opened by the particular process.
- root : symbolic link pointing to the directory which is the root file system for the particular process
- status : information about the process

Other Subdirectoriesin /proc

- /proc/self : link to the currently running process
- /proc/bus : contains information specific to the various buses available on the system
  - eg : for ISA, PCI, and USB buses, current data on each is available in /proc/bus/<bus type directory>
  - Individual bus directories, signified with numbers, contains binary files that refer to the various devices available on that bus
  - *devices* file : USB root hub on the motherboard:

Subdirectories (cont...)

- /proc/driver : specific drivers in use by kernel
  - rtc : output from the driver for the Real Time Clock
- /proc/fs : specific filesystem, file handle, inode, dentry and quota information
- /proc/ide : information about IDE devices
  - Each IDE channel is represented as a separate directory, such as /proc/ide/ide0 and /proc/ide/ide1
  - *drivers* file : version number of the various drivers
  - Device directories : data like cache, capacity, driver, geometry, media, model, settings

Subdirectories (cont...)

- /proc/irq : used to set IRQ to CPU affinity
  - smp_affinity : which CPUs handle that specific IRQ
- /proc/net : networking parameters and statistics
  - arp — kernel's ARP table. Useful for connecting hardware address to an IP address on a system.
  - dev — Lists the network devices along with transmit and receive statistics.
  - route — Displays the kernel's routing table.
- /proc/scsi : like /proc/ide it gives info about scsi devices

/proc/sys

- allows you to make configuration changes to a running kernel
- Changing a value within a /proc/sys file is done by the 'echo' command
- Any configuration changes made thus will disappear when the system is restarted

/proc/sys subdirectories

- /proc/sys/dev : provides parameters for particular devices on the system
  - cdrom/info : many important CD-ROM parameters
- /proc/sys/fs
- /proc/sys/kernel
  - acct — Controls the suspension of process accounting based on the percentage of free space available on the filesystem containing the log

/proc/sys subdirectories (cont....)

- ctrl-alt-del — Controls whether [Ctrl]-[Alt]-[Delete] will gracefully restart the computer using init (value 0) or force an immediate reboot without syncing the dirty buffers to disk (value 1).
- domainname — Allows you to configure the system's domain name, such as domain.com.
- hostname — Allows you to configure the system's host name, such as host.domain.com.
- threads-max — Sets the maximum number of threads to be used by the kernel, with a default value of 4095.

/proc/sys subdirectories (cont....)

- The random directory data related to generating random numbers for the kernel.
- panic — Defines the number of seconds the kernel will postpone rebooting the system when a kernel panic is experienced. By default, the value is set to 0, which disables automatic rebooting after a panic.

- /proc/sys/net
- /proc/sys/vm : facilitates the configuration of the Linux kernel's virtual memory (VM) subsystem

/proc File System Entries

- To use any of the procfs functions, you have to include the correct header file! #include <linux/proc_fs.h>
- struct proc_dir_entry* **create_proc_entry**(const char* *name*, mode_t *mode*, struct proc_dir_entry* *parent*);
  - This function creates a regular file with the name *name*, the mode *mode* in the directory *parent*.
  - To create a file in the root of the procfs, use NULL as *parent* parameter.
  - When successful, the function will return a pointer to the freshly created *struct proc_dir_entry*
  - *foo_file = create_proc_entry("foo", 0644, example_dir);*

Creating a Directory and a Symlink

- *struct proc_dir_entry** ***proc_mkdir****(const char* name, struct proc_dir_entry* parent);*
  - Create a directory *name* in the procfs directory *parent*.
- struct proc_dir_entry* **proc_symlink**(const char* *name*, struct proc_dir_entry* *parent*, const char* *dest*);
  - This creates a symlink in the procfs directory *parent* that points from *name* to *dest*. This translates in userland to ln -s *dest name*.

Removing an Entry

- void **remove_proc_entry**(const char* *name*, struct proc_dir_entry* *parent*);
  - Removes the entry *name* in the directory *parent* from the procfs.
  - Be sure to free the *data* entry from the struct proc_dir_entry before remove_proc_entry is called

Advantages & Disadvantages of the /proc File System

- Advantages
  - Coherent, intuitive interface to the kernel
  - Great for tweaking and collecting status info
  - Easy to use and program for
- Disadvantages
  - Certain amount of overhead, must use fs calls
    - Alleviated somewhat by sysctl() interface
  - User can possibly cause system instability

Outline

- Linux Kernel Programming Basic
- Introduction of the /proc File System
- **Module Program in /proc File System**

Creating the /proc file in modules (1)

read_write:

create file: /proc/net/hyperCard

```
struct proc_dir_entry *create_proc_entry(
const char *name,
mode_t mode,
struct proc_dir_entry *parent )

struct proc_dir_entry *entry = NULL;
entry = create_proc_entry("hyperCard",
S_IFREG|S_IRUGO|S_IWUSR, &proc_net);
if (!entry) {
printk(KERN_ERR "unable to create /proc/net/hyperCard\n");
return -EIO;
} else {
entry->read_proc = hypercard_proc_read;
entry->write_proc = hypercard_proc_write;
}
```

Creating the /proc file in modules (2)

```
struct proc_dir_entry {
unsigned int low_ino;
unsigned short namelen;
const char *name;
mode_t mode;
nlink_t nlink;
uid_t uid;
gid_t gid;
loff_t size;
struct inode_operations * proc_
const struct file_operations *
get_info_t *get_info;
struct module *owner;
struct proc_dir_entry *next, *p
void *data;
read_proc_t *read_proc;
write_proc_t *write_proc;
atomic_t count; /* use
int deleted; /* dele
void *set;
};
```

Creating the /proc file in modules (3)

read-only:

create file: /proc/test-file

```
struct proc_dir_entry *create_proc_read_entry
(
const char *name,                    → "test-file"
mode_t mode,                         → the access mode of the file (0444)
struct proc_dir_entry *base,         → the location of the file (NULL)
read_proc_t *read_proc,              → function called when the file is read
void *data                           → data used by read_proc function
);
```

Creating the /proc file in modules (4)

int (*read_proc)(char *page, char **start, off_t offset, int count, int *eof, void *data);

- **→page points to the buffer to write your data**
- **→start is used only if your data is beyond one page**
- **→offset means where to start from**
- **→count bytes to read**
- **→eof points to an integer that is set by the driver to indicate EOF**
- **→data driver-specific data pointer for internal bookkeeping, return the number of bytes actually placed in the page buffer**

Creating the /proc file in modules (5)

linux/proc_fs.h

struct proc_dir_entry *create_proc_read_entry(...)

```
struct proc_dir_entry {
unsigned int low_ino;
unsigned short namelen;
const char *name;
mode_t mode;
nlink_t nlink;
uid_t uid;
gid_t gid;
loff_t size;
struct inode_operations * proc_iops;
```

Creating the /proc file in modules (6)

```
const struct file_operations * proc_fops;
get_info_t *get_info;
struct module *owner;
struct proc_dir_entry *next, *parent, *subdir;
void *data;
read_proc_t *read_proc;
write_proc_t *write_proc;
atomic_t count; /* use count */
int deleted; /* delete flag */
void *set;
};
```

Example (1)

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
static int hello_call (char *buf, char **start, off_t off, int count, int *eof, void *data)
{
printk(KERN_INFO "This is a test for proc file.\n");
int len = sprintf(buf, "%s", "Message from a Linux kernel module.\n");
return len;
}
static int __init hello_init (void)
{
struct proc_dir_entry *entry;
entry = create_proc_read_entry ("hello", 0444, NULL, hello_call, NULL);
if(!entry)
return -1;
else{
Printk (KERN_INFO "Proc_read_entry created successfully.\n");
return 0;
}
}
```

上海交通大學

Example (2)

```
static void __exit hello_exit(void)
{
remove_proc_entry("hello", NULL);
printk(KERN_INFO "Goodbye.\n");
}
module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

```
struct proc_dir_entry *create_proc_read_entry
(
const char *name,
mode_t mode,
struct proc_dir_entry *base,
read_proc_t *read_proc,
void *data
);
```

Other Functions

- proc_mkdir creates a new directory.
- proc_mkdir_mode creates a new directory whose access mode can be explicitly specified.
- proc_symlink generates a symbolic link.
- remove_proc_entry deletes a dynamically generated entry from the proc directory.

```
struct proc_dir_entry *create_proc_read_entry
(
const char *name,
mode_t mode,
struct proc_dir_entry *base,
read_proc_t *read_proc,
void *data
);
struct proc_dir_entry *create_proc_entry
( const char *name,
mode_t mode,
struct proc_dir_entry *parent
);
```

Printk()

```
int printk(const char * fmt,…);
```

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
static int __init hello_init(void)
{
printk(KERN_INFO "Hello world\n");
return 0;
}
static void __exit hello_exit(void)
{
printk(KERN_INFO "Goodbye world\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

Output the text:

#dmesg | tail -5

loglevels:

KERN_EMERG = “<0>”
KERN_ALERT
KERN_CRIT
KERN_ERR
KERN_WARNING
KERN_NOTICE
KERN_INFO
KERN_DEBUG

Modules VS. Programs

Modules

- module_init() and module_exit() functions
- Avaialble functions
  - printk()
  - Kernel Space
  - Written in C (typically)

Programs

- main() function
- Avaialble functions
  - printf()
  - User Space
  - Written in C, C++, Java, etc.

Tips

- Modules can only use APIs exported by kernel and other modules
  - No libc
  - Kernel exports some common APIs
- Modules run in ring 0
  - Security holes
- Modules are part of kernel
  - Modules can control the whole system
  - As a result, damage the whole system
- Modules basically can't be written with C++
- Determine which part should be in kernel

Commands Frequently Used (1)

Create a patch

a) Assume under directory /root/work

b) Copy the source tree of kernel here, or untar the source codes.

Assume the tree is linux-2.6.17;

c) Copy the source tree by #cp -r linux-2.6.17 linux-2.6.17-fix;

d) Change files under linux-2.6.17-fix;

e) After changes are done; #cd /root/work

f) #diff -Nraup linux-2.6.17 linux-2.6.17-fix>patch_name.patch

g) If you once build kernel under your new tree,

pls. delete the obj by #make mrproper before run #diff

Apply a patch

a) Assume your working directory of new kernel is /root/work/new_tree;

b) #cd /root/work/new_tree

c) #patch -p1</root/work/patch_name.patch

Commands Frequently Used (2)

Create cross-index for code lookup: two approaches

a) Tags:

i. #cd /root/work/linux-2.6.17

ii. #make tags #you just need run it once

iii.When you look for a symbol/variable/definition,

run #vim -t definition_name.

b) cscope:

i. buildup cscope database: (Only need once)

1. #cd /root/work/linux-2.6.17

2. #find . -name "*.[chS]">files

3. #find . -name "*Makefile*">>files

4. #cscope -bk -f cc.db -I files

ii. Lookup symbol:

1. #cscope -d -f cc.db -p10

2. You will see a small menu. Query symbol/file/head_files